WALL-E & EVE's Signal: Interview Master Guide

Role: First-Year CSE Fresher | **Club:** VinnovateIT (AI/ML Subdomain) | **Duration:** 10 Minutes
Comprehensive technical preparation, spoken pitch scripts, line-by-line code walk, and 25 Q&As.

---

1. The Project in 60 Seconds

What is this project? It is an autonomous AI robot simulation built in pure Python. The robot, WALL-E, is placed in an unknown, obstacle-filled 2D grid world. Somewhere on this grid is EVE, who is emitting faint, noisy radio signals.

The Challenge: WALL-E has a limited battery (100 units). The sensors are noisy (bearing angles are off by up to 20 degrees, signal strength fluctuates, signals randomly drop out), and deceptive 'decoy' beacons try to fool him. If WALL-E runs out of battery before returning to a charger, he gets permanently stranded.

How it solves it: Instead of guessing, WALL-E uses **Active Bayesian Grid Filtering**. He maintains a 3D probability map over every possible cell and every possible status (active, low power, damaged). At every single step, he calculates **Expected Information Gain (EIG)** to choose the next best viewpoint, while a **Centralized Safe-Return Reserve Guard** guarantees he always has enough battery to reach the nearest charging station.

Real-Life Analogy: *Imagine playing 'Hot and Cold' in a pitch-black warehouse with walls. You have a noisy walkie-talkie that beeps louder when you get closer and points a faint arrow. Every step drains your phone battery, but there are power outlets on the walls. You don't just blindly run toward the beeps; you mathematically track where the sound is most likely coming from on a notepad, while always keeping enough battery to walk back to the nearest outlet.*

2. Spoken Pitches (Natural First-Year Student Voice)

The 30-Second Elevator Pitch:

"Hi! I built an autonomous Bayesian search agent called WALL-E that locates and diagnoses a lost robot, EVE, in a partially observed, obstacle-filled world. What makes it special is that it solves two hard problems together: it handles noisy, deceptive sensor signals using mathematical probability updates, and it strictly guarantees zero battery stranding using dynamic shortest-path return reserves. In 100 benchmark tests, it achieved a 98% search success rate while baselines struggled below 10%."

The 2-Minute Deep Pitch:

"Good morning! As a first-year student, I wanted to understand how real-world autonomous robots make decisions under extreme uncertainty without needing giant neural networks. So I implemented WALL-E & EVE's Signal in pure Python and NumPy.

The scenario is simple: WALL-E is in a 20x20 maze with battery limits. EVE is hidden somewhere, and her status could be active, low power, or damaged. When WALL-E scans, he gets noisy distance and bearing readings, occasional dropouts, and even spoofed decoy signals.

To solve this, I built three core systems:
First, a Bayesian Grid Filter that maintains a joint probability distribution over 1,200 possible states. Every sensor reading updates this belief using Bayes' Theorem.
Second, an Active Planner using Mutual Information. Instead of wandering randomly, WALL-E evaluates candidate viewpoints and moves where he expects to gain the most information.
Third, a Safe-Return Invariant Guard based on Breadth-First Search. Before executing any move or scan, the planner calculates the exact battery needed to reach the nearest charger with safety margins. If an action would strand him, it is automatically blocked.

Across 100 randomized test worlds, WALL-E localized EVE with an average error of just 0.3 cells and maintained a 0% stranding rate. I also built a trace replay verifier and full visual analytics pipeline."

3. How It Works Step by Step (System Architecture)

Here is the exact lifecycle of a simulation step, from input to output:

4. File-by-File Guide

What every file does in 1 to 2 lines, so you never get lost if asked about project structure:

File / Folder	Exact Responsibility in Codebase
src/walle/agent.py	Agent state machine (EXPLORE->TRACK->VERIFY->DECLARE), clue tracking, and decision safety wrapper.
src/walle/belief.py	3D Bayesian grid tensor B[y,x,k], likelihood multiplication, entropy calculation, and 90% credible regions.
src/walle/planner.py	Candidate viewpoint generation, Monte Carlo Expected Information Gain (EIG), and return reserve calculation.
src/walle/signal_model.py	Physical sensor model: inverse distance signal decay, angular noise, dropouts, and robust decoy mixture likelihood.
src/walle/world.py	Grid world generator: builds connected obstacle maps, places chargers, and hides EVE and clues.
src/walle/environment.py	Simulation loop: executes agent moves, checks wall bumps, deducts battery, and delivers sensor observations.
src/walle/runner.py	Mission runner: loads YAML, loops steps, enforces timeouts, logs telemetry, and compiles final metrics.
src/walle/viz.py	Visualization engine: renders summary panels, 2D paths, 3D space-time plots, belief heatmaps, and animated GIFs.
src/walle/verify.py	Independent trace verifier (CLI: walle verify) that replays mission logs to verify physics and energy conservation.

src/walle/benchmark.py	Generalization benchmark harness: runs 100 missions across 3 architectures, writes CSVs, and plots CDF/reliability charts.
src/walle/config.py	Strict dataclass schemas: validates all YAML parameters, rejects negative costs, and prevents bool-as-int bugs.
src/walle/observations.py	Data classes defining what the agent perceives (position, battery, signal reading, clues, status).
src/walle/actions.py	Data classes defining legal agent action types (MOVE_N/S/E/W, SCAN_SIGNAL, SCAN_AREA, RECHARGE, DECLARE).
src/walle/baselines.py	Comparison baseline agents (RandomAgent and GreedyBearingAgent) to benchmark against our BayesPlanner.
src/walle/geometry.py	Pure math helpers: Euclidean distance, angle wrapping (-180 to +180), and grid adjacency.
scripts/calibration_study.py	Evaluates statistical calibration across 200 seeds; outputs calibration.md and reliability diagram.
scripts/ablation_decoys.py	50-seed paired experiment showing the robust agent (+30% success) vs naive agent under decoys.
scripts/manual_play.py	Interactive terminal game allowing a human to play as WALL-E using w/a/s/d keys.

5. Tech Stack: Tools, Libraries & Alternatives

Know what each tool does, why we chose it, and one simple alternative:

Tool / Library	What It Is	Why We Used It Here	One Alternative
Python 3.10+	Programming language	Readable, standard in AI/ML, rich built-in libraries (dataclasses, typing).	C++ or Julia
NumPy	Numerical tensor library	Vectorized 3D array math for exact Bayesian updates; fast pure-Python convolution.	PyTorch or SciPy
Matplotlib	Plotting & chart library	Generates publication-quality 6-panel summary plots, CDF curves, and heatmaps.	Seaborn or Plotly
Pillow (PIL)	Image manipulation	Stitches per-step visualization frames into a smooth animated GIF (run.gif).	OpenCV
PyYAML	Config parser	Human-readable mission configuration files (grid size, battery, noise).	JSON or TOML
Pytest	Testing framework	Automated unit tests, 200-seed fuzz testing, and AST separation checking.	Unittest

6. Beginner Glossary of Core Technical Terms

Clear, simple definitions with real-life analogies:

Prior Probability: Your belief before seeing any new evidence. *Analogy: Before rolling a die, you assume every face has a 1/6 chance. That is your prior.*

Likelihood: How probable an observation is, assuming a specific hypothesis is true. *Analogy: If it is raining outside, seeing wet footprints is very likely; if sunny, wet footprints are unlikely.*

Posterior Probability: Your updated belief after combining the prior with the new observation. *Analogy: Seeing wet footprints shifts your belief that it rained from 10% to 90%.*

Bayes' Theorem: The mathematical formula: Posterior = (Prior * Likelihood) / Total Evidence. *Analogy: The scientific rule for updating your mind when new facts arrive.*

Belief Tensor: A 3D numerical grid B[y, x, k] storing probability mass for each cell and status. *Analogy: Think of a spreadsheet where every grid square has 3 numbers: active %, low-power %, damaged %.*

MAP (Maximum A Posteriori): The single cell and status that currently has the highest probability. *Analogy: Picking the MVP player with the highest score on the leaderboard.*

Shannon Entropy: A mathematical measure of uncertainty, measured in bits. *Analogy: If you have no clue where EVE is, entropy is high (~10 bits). When you pinpoint her, entropy drops to near 0.*

Expected Information Gain (EIG): How many bits of uncertainty you expect to eliminate by taking an action. *Analogy: Deciding whether to open Door A or Door B based on which one reveals the most answers.*

Return Reserve: The minimum battery required to safely reach the nearest charger from your position. *Analogy: Driving a car and knowing you must reach a petrol pump before the fuel light hits zero.*

Breadth-First Search (BFS): An algorithm that finds the shortest path on a grid by exploring ring by ring. *Analogy: Like ripples spreading outward in water until they touch the nearest charging station.*

Decoy / Spoofing: A fake signal designed to mislead the searcher. *Analogy: A prankster shouting 'over here!' from behind a tree when the real person is elsewhere.*

Robust Mixture Likelihood: A formula adding a small uniform probability floor (eps=0.15) to prevent zero-lock. *Analogy: Never saying '100% impossible' so a single fake signal cannot completely trick you.*

Credible Region (90%): The smallest set of grid cells that contains 90% of the total probability mass. *Analogy: Drawing a circle on a treasure map and being 90% confident the chest is inside.*

7. The 25 Most Likely Interview Questions & Answers

Grouped by category, with spoken answers, simple explanations, and follow-up defenses:

Q1 (Basic): What is the core problem this project solves?

Say aloud: ***"It solves autonomous search under uncertainty with strict resource limits: finding a lost target using noisy, deceptive sensors without running out of battery."***

The concept behind it: Most search algorithms either ignore noise or run out of power. WALL-E combines Bayesian probability filtering with a guaranteed safe-return battery guard.

Likely follow-up: *Follow-up: Why not just use Dijkstra or A* directly?*
Follow-up answer: *Dijkstra and A* require knowing the destination coordinates in advance. In our problem, the destination is completely hidden and must be inferred from noisy sensors.*

Q2 (Basic): What was your specific role in this project?

Say aloud: ***"I designed the problem specifications, configured the mission environments, verified the algorithms against edge cases, and audited the code for zero-cheating compliance."***

The concept behind it: As a fresher working with Antigravity, my focus was ensuring architectural correctness: making sure the agent never peeks at ground truth and strictly satisfies the safe return invariant.

Likely follow-up: *Follow-up: How did you ensure the agent didn't cheat?*
Follow-up answer: *We wrote an automated AST code scanner and runtime proxy interceptor (test_separation.py) that strictly forbids the agent from accessing private world attributes.*

Q3 (Basic): How is EVE's status represented, and why does it matter?

Say aloud: ***"EVE has three operational states: active, low power, and damaged. Each state produces different signal strengths (1.0, 0.6, 0.35) and different dropout probabilities (5%, 25%, 50%)."***

The concept behind it: The status affects how easily EVE can be heard. A damaged EVE emits faint signals that drop out half the time, requiring WALL-E to get closer before making a confident diagnosis.

Likely follow-up: *Follow-up: Can WALL-E declare the right location but the wrong status?*
Follow-up answer: *Yes, that counts as a failure. A mission is only successful if BOTH the location is within 2 cells AND the operational status is correctly identified.*

Q4 (Basic): What are the charging stations, and how do they work?

Say aloud: ***"Charging stations are dedicated cells on the grid. When WALL-E executes the RECHARGE action while on a station, he gains +20 battery units over 2 time ticks, up to capacity."***

The concept behind it: Chargers allow WALL-E to operate indefinitely on large maps as long as he returns to recharge before his battery drops below the return reserve.

Likely follow-up: *Follow-up: Can WALL-E recharge anywhere?*
Follow-up answer: *No. Recharging off a charger is rejected by the environment as an invalid action, consuming ticks without giving energy.*

Q5 (Basic): What are clues, and how do they differ from signal scans?

Say aloud: ***"Signal scans are long-range radio readings that can be done anywhere. Clues are physical objects hidden in specific cells that are discovered only when WALL-E performs an AREA_SCAN."***

The concept behind it: Clues give high-accuracy evidence (bearing hints, range hints, or status diagnostic logs) to resolve ambiguities.

Likely follow-up: *Follow-up: Does WALL-E know where the clues are placed?*
Follow-up answer: *No, clue coordinates are completely hidden from the agent. He only discovers them when his area scan radius covers that cell.*

Q6 (Technical): How does the Bayesian Grid Filter update beliefs?

Say aloud: ***"It treats each grid cell and status as a hypothesis. When a reading arrives, it multiplies the prior probability by the likelihood of getting that reading, and renormalizes so the sum equals 1."***

The concept behind it: This is exact discrete Bayes: Posterior[y,x,k] = Prior[y,x,k] * Likelihood(reading | y,x,k) / Evidence.

Likely follow-up: *Follow-up: Why not use a continuous Kalman Filter?*
Follow-up answer: *A Kalman Filter assumes a single Gaussian 'bell curve'. But on a grid with obstacles and decoys, belief is multimodal (it has multiple peaks in different rooms). A discrete grid filter handles any shape.*

Q7 (Technical): How does the Safe-Return Invariant work mathematically?

Say aloud: ***"Before every action, the planner computes: battery_after >= shortest_path_to_charger * move_cost * (1 + margin_fraction) + margin_flat. If false, the action is blocked."***

The concept behind it: We run a Breadth-First Search (BFS) from the proposed position to all charger cells to find the exact shortest obstacle-free path distance.

Likely follow-up: *Follow-up: What happens if WALL-E is already in an unsafe state?*
Follow-up answer: *The safety guard immediately intercepts decision-making, clears all planned exploration, and commands the shortest-path move toward the nearest charger.*

Q8 (Technical): How does WALL-E decide what action to take next?

Say aloud: ***"He evaluates up to 25 candidate actions, estimates their Expected Information Gain (EIG) using Monte Carlo sampling, divides by action cost, and picks the highest safe scorer."***

The concept behind it: This is 'Next-Best-Viewpoint' active sensing. Actions that clarify the most uncertainty for the least energy get prioritized.

Likely follow-up: *Follow-up: Why divide EIG by cost?*
Follow-up answer: *Because a scan with radius 3 gives high information but costs 8 battery, while radius 1 costs 4. Dividing by cost gives the 'bang for the buck' efficiency.*

Q9 (Technical): How does the system handle decoys and spoofed signals?

Say aloud: ***"We use a robust mixture likelihood with a uniform floor (eps=0.15) and a 3-attempt confirmation state machine that forces triangulation from orthogonal viewpoints."***

The concept behind it: If a signal contradicts the current confident belief by >3 standard deviations, it is flagged as suspicious. The agent moves 45 to 135 degrees away to verify it from a different angle before declaring.

Likely follow-up: *Follow-up: What is the benefit in numbers?*
Follow-up answer: *In our 50-seed ablation study, the robust agent achieved 90% success on hard decoy missions, compared to only 60% for a naive agent.*

Q10 (Technical): What are the different area scan radii and how are they chosen?

Say aloud: ***"WALL-E can scan radius 1 (costs 4 battery), radius 2 (costs 6 battery), or radius 3 (costs 8 battery). The planner evaluates all 3 and picks based on battery budget and unscanned cells."***

The concept behind it: Radius 1 is chosen when battery is tight; radius 3 is chosen in open spaces when battery is plentiful to sweep wide areas.

Likely follow-up: *Follow-up: Does WALL-E rescan the same area repeatedly?*
Follow-up answer: *No. The agent maintains a spatial memory mask of scanned cells. Scanning already-cleared cells yields 0 information gain, naturally suppressing redundant scans.*

Q11 (AI/ML): What is Shannon Entropy and why is it used?

Say aloud: ***"Entropy measures how uncertain the agent is, in bits. High entropy means probability is spread flat across the whole map; low entropy means it is concentrated on one spot."***

The concept behind it: Formula: H = - sum(p * log2(p)). It gives the planner a single number to track search progress. When entropy drops below 2 bits, WALL-E knows he has narrowed down the search.

Likely follow-up: *Follow-up: What is the theoretical maximum entropy on a 20x20 grid with 3 statuses?*
Follow-up answer: *There are 20 * 20 * 3 = 1,200 states. Uniform log2(1200) is approximately 10.23 bits of entropy.*

Q12 (AI/ML): What is Expected Information Gain (EIG)?

Say aloud: ***"EIG is the expected drop in entropy before taking an action: the Mutual Information between the target location and the future sensor reading."***

The concept behind it: Mathematically: I(X; Y) = E[D_KL(P(X|Y) || P(X))]. By Gibbs' inequality, this is mathematically guaranteed to never be negative.

Likely follow-up: *Follow-up: How do you compute it if you haven't taken the action yet?*
Follow-up answer: *We simulate 25 possible sensor readings using Monte Carlo sampling from our current belief, compute the hypothetical posterior for each, and take the weighted average.*

Q13 (AI/ML): What is a 90% Credible Region?

Say aloud: ***"It is the Bayesian version of a confidence interval: the smallest set of cells whose cumulative posterior probability sums to at least 0.90."***

The concept behind it: In classical statistics, a confidence interval is about repeated experiments. In Bayesian statistics, a credible region directly means: 'There is a 90% chance the target is inside this box.'

Likely follow-up: *Follow-up: How do you verify if the credible region is accurate?*
Follow-up answer: *Through calibration testing across 200 random missions: we checked if EVE actually fell inside the 90% region at least 90% of the time. Our empirical result was 93%.*

Q14 (AI/ML): What is Kullback-Leibler (KL) Divergence?

Say aloud: ***"KL divergence measures how much one probability distribution differs from another, expressed in bits."***

The concept behind it: Formula: D_KL(P || Q) = sum(P * log2(P / Q)). We use it to measure the exact amount of information a new clue or initial signal adds relative to a uniform prior.

Likely follow-up: *Follow-up: What if P and Q are identical?*
Follow-up answer: *If P equals Q everywhere, the log ratio is log2(1) = 0, so the KL divergence is exactly 0.0 bits.*

Q15 (AI/ML): What is the difference between this Bayesian approach and Reinforcement Learning (RL)?

Say aloud: ***"RL requires thousands of training episodes to learn a policy via trial and error. Bayesian active planning requires ZERO training; it computes optimal decisions on the fly using exact probability theory."***

The concept behind it: For safety-critical tasks with battery constraints, Bayesian planning gives formal guarantees (0% stranding) that neural RL networks cannot guarantee without extensive guardrails.

Likely follow-up: *Follow-up: When would RL be better?*
Follow-up answer: *If the world had complex continuous physics, visual camera pixels, or unknown dynamic opponents, RL with deep neural networks could learn features that simple grid filters cannot.*

Q16 (Code): Explain Belief.update() in belief.py line by line.

Say aloud: ***"It takes a 3D likelihood array L, multiplies it element-wise with our prior tensor self._b, calculates total evidence S = np.sum(new_b), and normalizes new_b / S."***

The concept behind it: If total evidence is zero (impossible observation), it preserves the prior to prevent division by zero. It also computes exact KL divergence to record the information bits gained.

Likely follow-up: *Follow-up: What is the shape of self._b?*
Follow-up answer: *It is (height, width, 3), representing (y, x, status). For a 20x20 grid, it has exactly 1,200 floating point numbers.*

Q17 (Code): Explain return_reserve() in planner.py line by line.

Say aloud: ***"It calls bfs_distance_to_nearest() to find the shortest obstacle-free cell distance to the closest charger, multiplies by move cost (1), applies (1 + margin_fraction), and adds margin_flat."***

The concept behind it: Code: d = bfs_distance(pos, chargers, walls); reserve = int(ceil(d * cost * (1.0 + frac))) + flat. If already on a charger, the reserve is strictly 0.

Likely follow-up: *Follow-up: Why use ceil()?*
Follow-up answer: *Because battery units are discrete integers. Using ceil ensures we round up, never underestimating the energy needed to survive.*

Q18 (Code): Explain build_signal_likelihood() in signal_model.py.

Say aloud: ***"It creates a 3D likelihood tensor across all (y, x, status). For each cell, it calculates distance d to agent, expected strength A_k * d0 / (d + d0), and computes normal PDF values for strength and bearing."***

The concept behind it: It handles dropouts by assigning status-specific dropout probabilities uniformly across cells, and handles co-location (d=0) where bearing is physically undefined.

Likely follow-up: *Follow-up: How is the decoy spoof rate blended in?*
Follow-up answer: *L_robust = (1 - eps) * L_true + eps * (1.0 / num_free_cells). This uniform mixture prevents any cell from having absolute 0 probability.*

Q19 (Code): Explain the decoy confirmation state machine in agent.py.

Say aloud: ***"When an observation has >3 sigma angular error, agent sets pending_confirmation=True. Candidate viewpoints with orthogonal angles (45-135 deg) get a 2.5x utility bonus."***

The concept behind it: If 3 attempts from distinct cells fail to confirm, the reading is permanently marked confirmed_decoy. Crucially, the agent is strictly prohibited from declaring while pending_confirmation is True.

Likely follow-up: *Follow-up: Why must the confirmation be from a distinct cell?*
Follow-up answer: *Because scanning from the exact same cell would just repeat the same directional perspective, causing circular self-reinforcement of the fake signal.*

Q20 (Code): Explain walle verify in verify.py.

Say aloud: ***"It loads trace.jsonl, report.json, and config.yaml, and replays every action step independently without calling the agent code."***

The concept behind it: It verifies: (1) moves are 4-connected and do not cross walls, (2) battery decreases by exact cost table values, (3) recharge only happens on chargers, and (4) battery >= reserve at every step.

Likely follow-up: *Follow-up: Why is this important?*
Follow-up answer: *It proves the simulation results are 100% authentic and mathematically honest. Evaluators can verify any run without trusting our word.*

Q21 (Challenges): What was the hardest bug you encountered and fixed?

Say aloud: ***"In runner.py, min_battery_margin was comparing post-step battery against pre-step reserve. When moving toward a charger, battery dropped by 1 but reserve also dropped by 1, falsely looking like a margin decrease."***

The concept behind it: We fixed it to evaluate battery against the post-step reserve at the new position. That reconciled the accounting and proved zero invariant violations.

Likely follow-up: *Follow-up: How was this caught?*
Follow-up answer: *By running our 200-seed Monte Carlo fuzz tests and the trace replay verifier.*

Q22 (Challenges): What are the current limitations of this project?

Say aloud: ***"It operates on a discrete 2D grid with known wall locations. In real robotics, walls have sensor noise (SLAM is needed) and motion is continuous (angles, velocity, wheel slip)."***

The concept behind it: Also, while 20x20 runs in milliseconds, scaling to 1000x1000 would require particle filtering or octree spatial partitioning rather than dense 3D tensors.

Likely follow-up: *Follow-up: What would you improve next if you had another month?*
Follow-up answer: *I would implement Particle Filtering (Monte Carlo Localization) to handle continuous coordinates and test multi-agent cooperation (two WALL-Es searching together).*

Q23 (Challenges): What if EVE moves around?

Say aloud: ***"We already implemented and tested discrete random walk and Gaussian diffusion mobility in belief.predict() using pure NumPy wall-aware separable convolution."***

The concept behind it: At each step, probability mass diffuses slightly to neighbor cells, conserving total mass to 10^-12 while widening belief until a fresh scan tightens it.

Likely follow-up: *Follow-up: How do you handle target motion around obstacles?*
Follow-up answer: *The diffusion kernel is strictly wall-aware: mass that would diffuse into obstacle cells is truncated and renormalized across free neighbor cells.*

Q24 (Pressure): Did you build this yourself or did an AI build it?

Say aloud: ***"Antigravity helped generate boilerplate, run large automated test suites, and format documentation. But I drove the architecture, debugged the core safety invariants, and understand every equation and module."***

The concept behind it: I can walk you through any function in this codebase right now line by line, explain the mathematical derivations, and modify any parameter live in front of you.

Likely follow-up: *Follow-up: What is one architectural decision you personally made?*
Follow-up answer: *Choosing exact mutual information EIG instead of simple entropy differences. Simple entropy differences can be negative on unnormalized samples; mutual information is mathematically guaranteed to be non-negative.*

Q25 (Pressure): Why should VinnovateIT select a first-year student like you?

Say aloud: ***"Because I don't just import high-level libraries and treat AI as a black box. I took the initiative to learn core mathematical fundamentals—Bayesian inference, information theory, state machines, and formal safety verification."***

The concept behind it: I have demonstrated engineering rigor: 139 passing unit tests, automated fuzzing, and production-grade documentation. I am passionate, eager to learn from seniors, and ready to contribute to club projects from Day 1.

Likely follow-up: *Follow-up: Which domain projects in VinnovateIT are you most excited about?*
Follow-up answer: *Autonomous robotics, reinforcement learning, and applying probabilistic AI to real-world edge devices.*

8. Honest AI Disclosure & Defensibility Guide

VinnovateIT interviewers respect honesty and deep technical understanding far more than pretending you wrote 10,000 lines from scratch in Notepad. Here is how to answer with complete confidence:

When asked: "Did you use AI to build this?"

Answer with poise:
"Yes, absolutely. I used Google Antigravity as an advanced AI pair-programming assistant, much like professional engineers use GitHub Copilot or cursor today. However, I didn't treat it as a code generator that I blindly copy-pasted.

I directed the design: defining the Bayesian belief update equations, enforcing strict agent/environment separation so WALL-E cannot cheat, and debugging the return reserve invariants when edge cases failed.

I can explain every file, write tests for any module, and explain the math behind the code. To me, AI is a force multiplier for learning complex concepts faster."

9. 10-Minute Interview Time Allocation

How to manage your 10 minutes so you don't get cut off before showing your best work:

Time Window	Stage	Goal & Action
0:00 - 1:30 (1.5 min)	Introduction & Pitch	Introduce yourself, give the 60-second pitch, state the problem and core Bayesian solution.
1:30 - 4:30 (3.0 min)	Live Demonstration	Run walle run --ascii in terminal. Show real-time belief convergence and open summary.png.

4:30 - 8:30 (4.0 min)	Technical Q&A;	Answer questions on Bayesian updates, BFS safe-return reserve, and decoy handling.
8:30 - 10:00 (1.5 min)	Closing & Why VinnovateIT	Share your learnings, explain your passion for AI/ML, and thank the interviewers.

10. Last-Minute Interview Cheat Sheet

15 Key Facts, Metrics & Formulas to Memorize:

1. Grid Dimensions: Default 20x20 grid with 4-connected moves (North, South, East, West).
2. State Space: 20 x 20 x 3 = 1,200 joint states (X, Y, Status: active, low_power, damaged).
3. Battery & Costs: 100 capacity. MOVE costs 1; SCAN_SIGNAL costs 2; SCAN_AREA costs 2+2r; RECHARGE gives +20.
4. Signal Physics: Amplitude A_k in {1.0, 0.6, 0.35}. Strength = A_k * d0 / (d + d0) with d0=6.0 cells.
5. Sensor Noise: Bearing error sigma_theta = 8 deg + 2 deg * distance. Signal strength noise sigma_s = 0.05.
6. Dropouts: Probability of missed signal: Active = 5%, Low Power = 25%, Damaged = 50%.
7. Decoys & Spoofing: Hard mission has 2 decoys and 15% spoof chance. Robust mixture uses uniform floor eps=0.15.
8. Safe Return Guarantee: battery_after >= bfs_distance_to_nearest_charger * 1 * (1 + 0.10) + 3.
9. Benchmark Performance: BayesPlanner achieves 98.0% success vs 6.0% for Greedy Bearing and 3.0% for Random.
10. Stranded Rate: Exactly 0.0% stranded across all 100 benchmark missions and 200 fuzz tests.
11. Localization Accuracy: 0.30 cells mean error for BayesPlanner (median 0.00 cells).
12. Information Gain: Shannon Mutual Information I(X;Y) = E[D_KL(P(X|Y) || P(X))] >= 0 bits.
13. Credible Coverage: 93.0% empirical coverage for nominal 90% credible regions across 200 calibration seeds.
14. Verification Command: python -m walle verify outputs/mission_medium_seed7 audits physics & battery step by step.
15. Zero Cheating: Environment hides EVE pos/status; audited by AST scanner and runtime attribute proxy.

11. 'Learn Tonight': The 5 Must-Know Concepts

If you only have time to understand 5 things tonight, master these 5 simple explanations:

1. Bayes' Theorem in 5 Lines

Bayes' theorem is how an AI updates its beliefs when new clues arrive.
You start with a Prior (what you believed before).
You multiply by the Likelihood (how well the clue matches that spot).
You divide by Evidence so all probabilities sum to 1.0 (100%).
The result is the Posterior: your new, smarter map of where EVE is.

2. Safe-Return Reserve in 5 Lines

WALL-E must never die in the field from battery depletion.
Breadth-First Search (BFS) finds the exact step count to the nearest charger around walls.
Reserve = (Steps * 1) + 10% safety buffer + 3 emergency units.
Before any step, WALL-E asks: 'Will my battery after this action stay above the reserve?'
If no, exploration is cancelled and WALL-E retreats directly to recharge.

3. Shannon Entropy in 5 Lines

Entropy measures total confusion or ignorance, measured in bits.
If EVE could be in any of the 1,200 states equally, entropy is maximum (~10 bits).
Every scan eliminates unlikely cells and lowers entropy.
When entropy drops below 2 bits, WALL-E has narrowed EVE down to a tiny neighborhood.
When entropy is near 0, WALL-E is confident enough to declare the mission finished.

4. Expected Information Gain (EIG) in 5 Lines

Instead of moving blindly, WALL-E thinks ahead one step.
He imagines: 'If I scan from Position A, what readings might I get?'
He simulates 25 possible readings and calculates how much entropy would drop.
This expected drop is the Expected Information Gain.
WALL-E picks the candidate that delivers the most information per battery spent.

5. Decoy Triangulation in 5 Lines

A decoy is a fake radio transmitter placed to trick WALL-E into wasting power.
If a signal points 90 degrees away from where EVE was already confirmed, it is flagged suspicious.
WALL-E does not declare immediately; he schedules a confirmation scan.
He deliberately moves 45 to 135 degrees to the side to view it from a right angle.
If the fake signal doesn't line up from both angles, it gets discounted as a confirmed decoy.